Gen
AI

Module - 2

Chunking, Embeddings, VectorDB & Tools

-

muhammadosama.hq@gmail.com

Gen
AI

Topics to be Covered

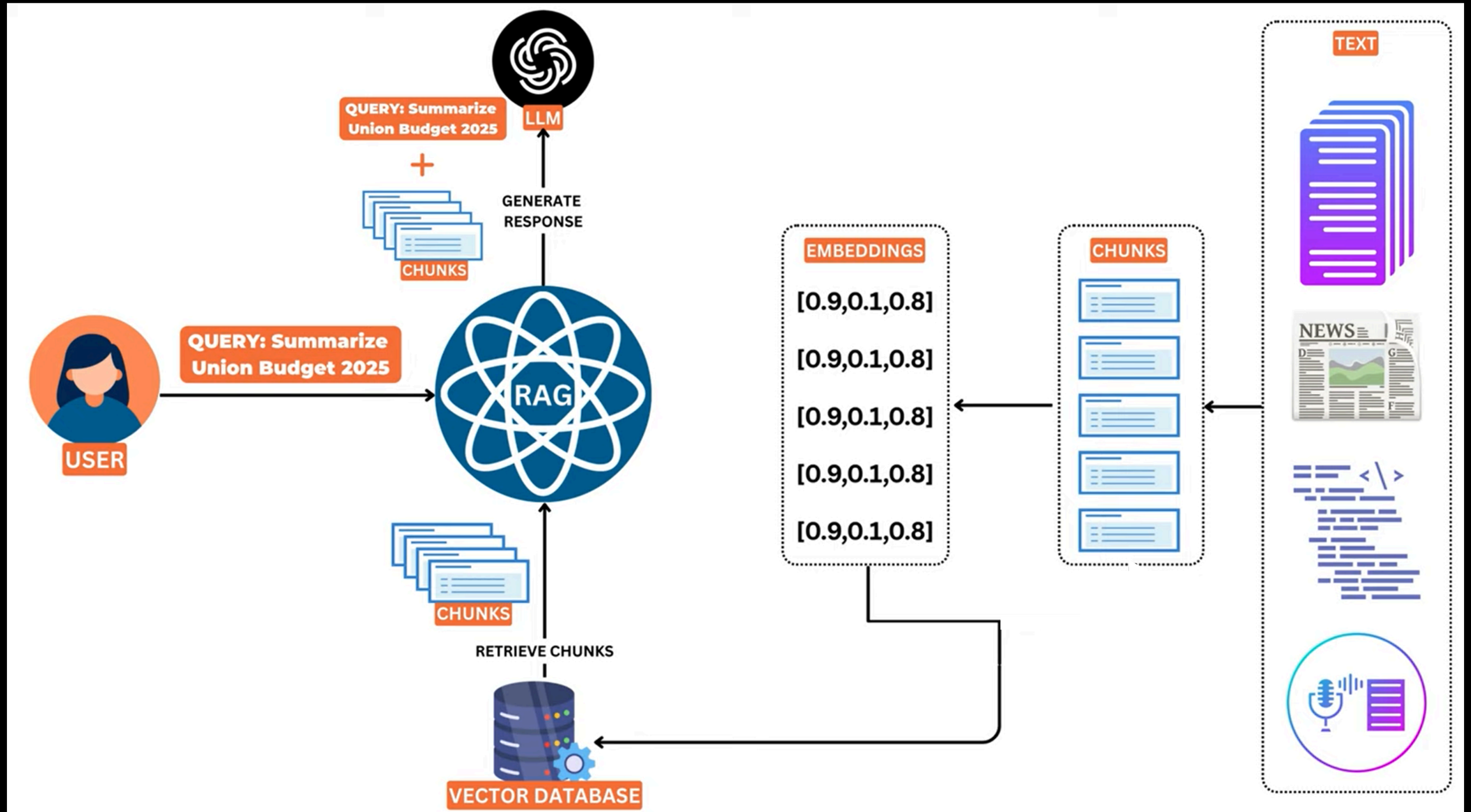
- Data Loaders
- Splitters
- Embeddings Model
- Vector Database
- Tools

Gen
AI

RAG on Custom Knowledge Base

Gen
AI

RAG Architecture



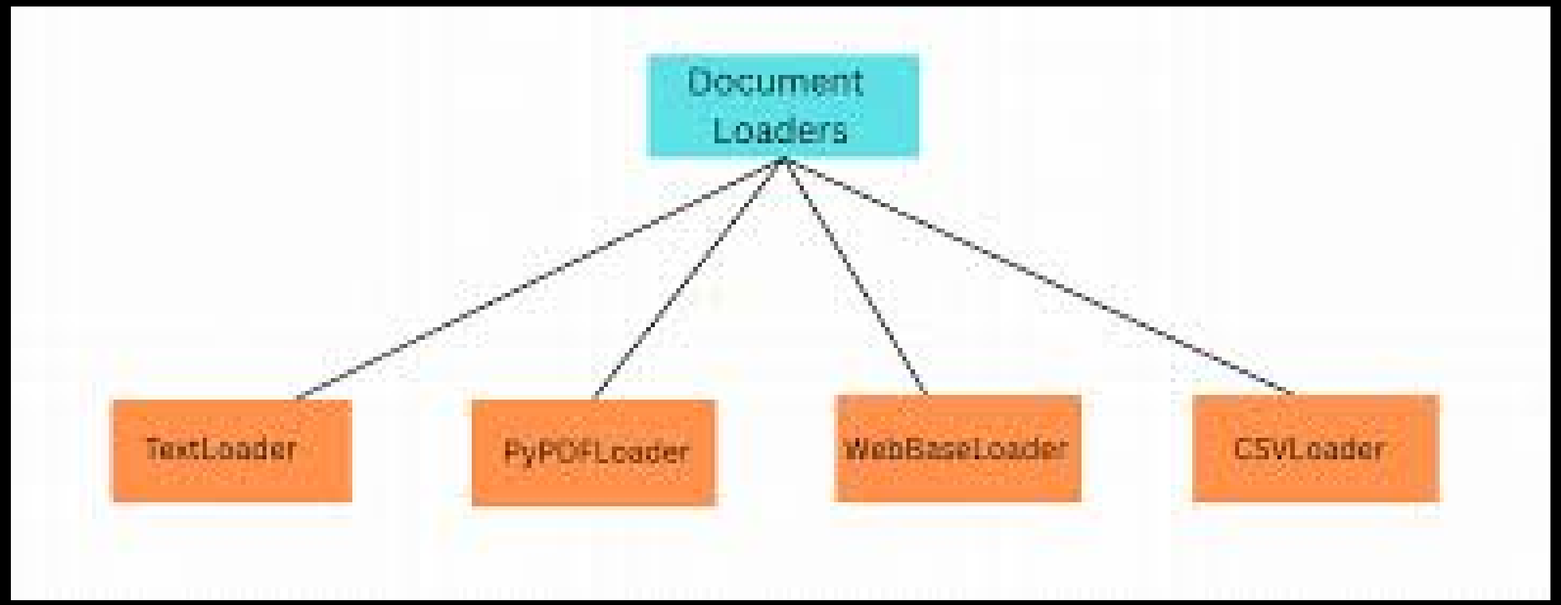
Gen
AI

Data Loaders

Gen
AI

What is Data Loader?

In LangChain, It is a component to load the data.



Gen
AI

Text Loader

```
from langchain_community.document_loaders import TextLoader
loader = TextLoader("few_shot_prompt.txt")

documents = loader.load()
for doc in documents:
    print(f"Content: {doc.page_content}")
    print(f"Metadata: {doc.metadata}")
```

Gen
AI

PDF Loader

```
# PDF Loader
from langchain_community.document_loaders import PyPDFLoader
loader = PyPDFLoader("./docs/IndustrialAI_Project.pdf")

pages = loader.load()
for p in pages:
    print(f"Content: {p.page_content}")
    print(f"Metadata: {p.metadata}")
```

Gen
AI

Upload A Directory – PDF

```
# Directory Loader
from langchain_community.document_loaders import DirectoryLoader, PyPDFLoader

loader = DirectoryLoader("./docs", glob="*.pdf", loader_cls=PyPDFLoader)

docs = loader.load()
for p in pages:
    print(f"Content: {p.page_content}")
    print(f"Metadata: {p.metadata}")
```

Gen
AI

Upload A Directory – Markdown → 1

```
# Directory Loader
from langchain_community.document_loaders import (
    DirectoryLoader,
    UnstructuredMarkdownLoader
)

loader = DirectoryLoader(
    path="./markdown_docs",
    glob="**/*.md",
    loader_cls=UnstructuredMarkdownLoader,
    recursive=True # Set to True to load files from subdirectories as well
)
```

Gen
AI

Upload A Directory – Markdown → 2

```
docs = loader.load()
print(f"Loaded {len(docs)} documents.")
if docs:
    print("\nSnippet of the first document:")
    print(docs[0].page_content[:500])
    print("\nMetadata of the first document:")
    print(docs[0].metadata)
```

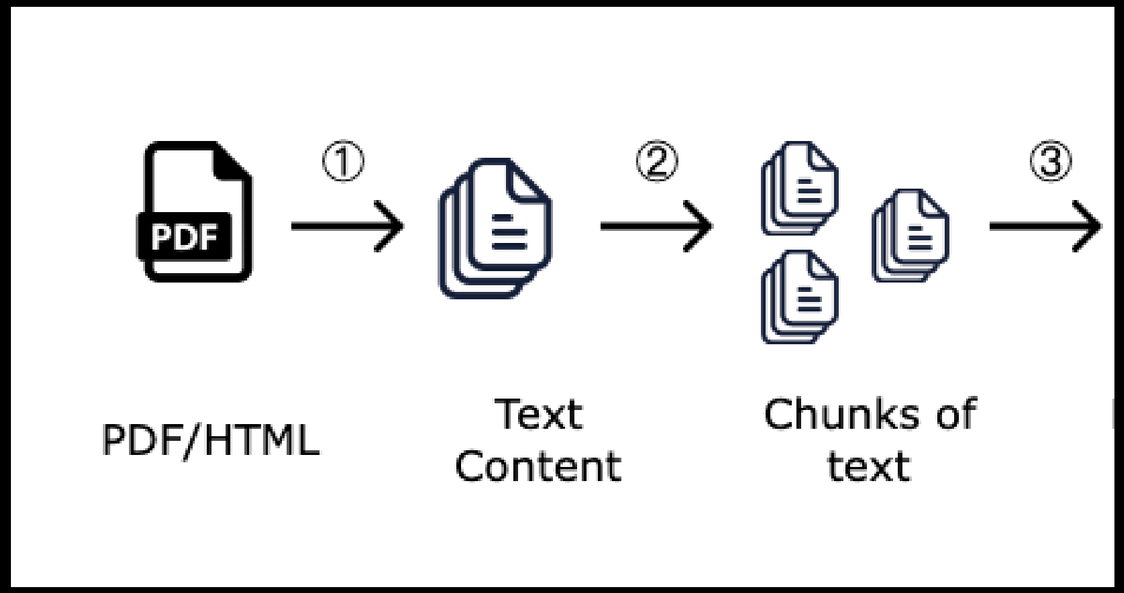
Gen
AI

Chunking

Gen
AI

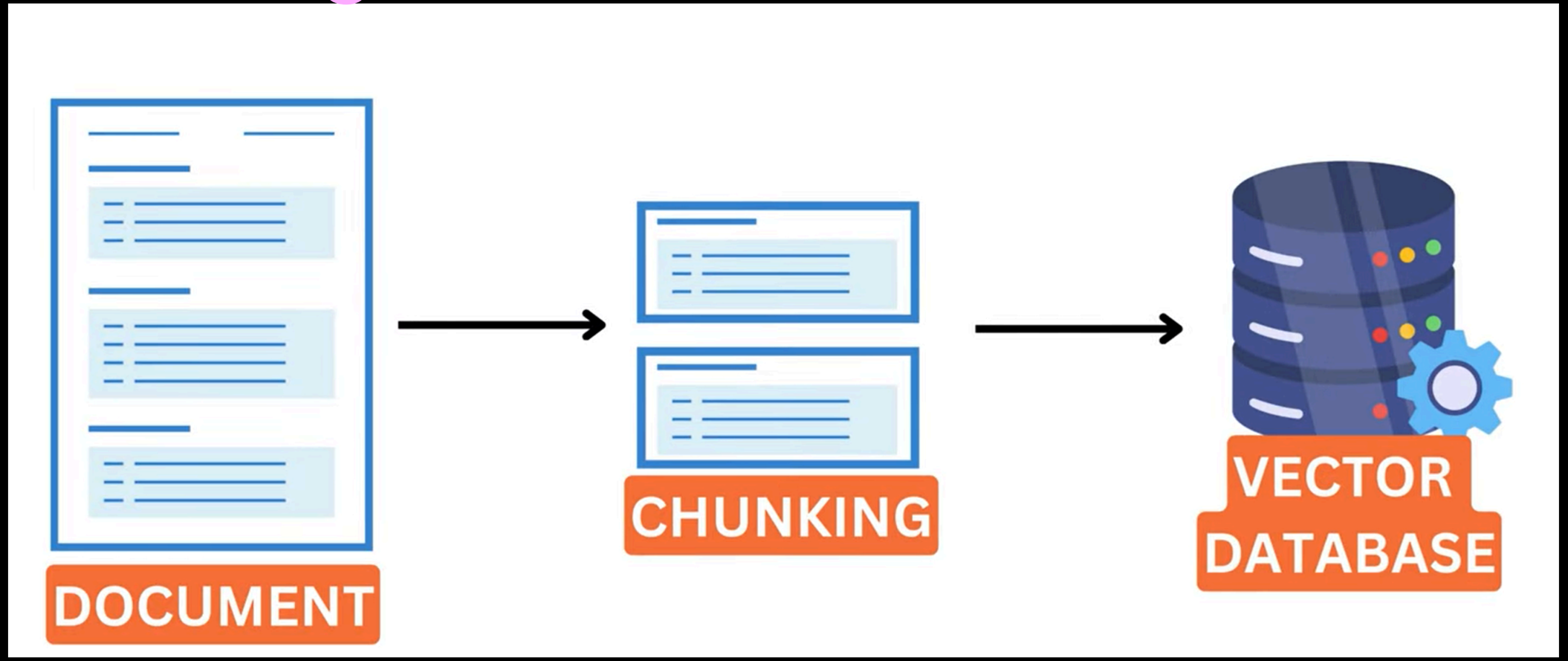
What is Chunking?

Chunking means breaking a large piece of text into smaller, manageable pieces called chunks.



Gen
AI

Chunking Visualization



Gen
AI

Why Chunking?

Following are the reasons due to we implement chunking:

- More relevant retrieval
- Better LLM context

Gen
AI

Case Study: Impact of Improper Chunking

A fintech company nearly lost a major deal because their AI chatbot provided incorrect legal advice. When asked about indemnification clauses in an NDA, their system confidently stated that “party A fully indemnifies party B” — omitting the critical exception clause that appeared in the next chunk. The contract actually read “party A indemnifies party B, except as provided in section 12.3,” but a poor chunking strategy had split this sentence across two separate pieces.

Gen
AI

Chunking Strategies

Following are the chunking strategies that we will explore:

- Text-based Chunking
- Document-based Chunking
- Contextual Chunking by Anthropic

To Learn More About Chunking Strategies (Splitters) in Langchain Visit:
https://docs.langchain.com/oss/python/integrations/splitters

Gen
AI

Text-based Chunking → RecursiveCharacterTextSplitter

```python
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Create a recursive text splitter
recursive_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,                    # Target chunk size in characters
    chunk_overlap=100,                 # Overlap between chunks
    separators=[
        "\n\n",      # First, try to split on double newlines (paragraphs)
        "\n",        # Then single newlines
        ". ",        # Then sentences
        ", ",        # Then clauses
        " ",         # Then words
        ""           # Finally, characters (last resort)
    ]
)
```

Gen
AI

Text-based Chunking → RecursiveCharacterTextSplitter

```
# Sample structured document
structured_document = """
Place your text to chunk here
"""
# Split the document recursively
recursive_chunks = recursive_splitter.split_text(structured_document)

# Display the chunks with their characteristics
for i, chunk in enumerate(recursive_chunks):
    print(f"Chunk {i + 1} (length: {len(chunk)} chars):")
    print(chunk)
    print("-" * 60)
```

Gen
AI

Document-based Chunking → Language Splitter

```
from langchain_text_splitters import (
    Language,
    RecursiveCharacterTextSplitter,
)

PYTHON_CODE = """
def hello_world():
    print("Hello, World!")

# Call the function
hello_world()
"""
```

Gen
AI

Document-based Chunking → Language Splitter

```
# Default Seperators
# ['\nclass ', '\ndef ', '\n\tdef ', '\n\n', '\n', ' ', '']

python_splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.PYTHON, chunk_size=50, chunk_overlap=0
)
python_docs = python_splitter.create_documents([PYTHON_CODE])
print(python_docs)
```

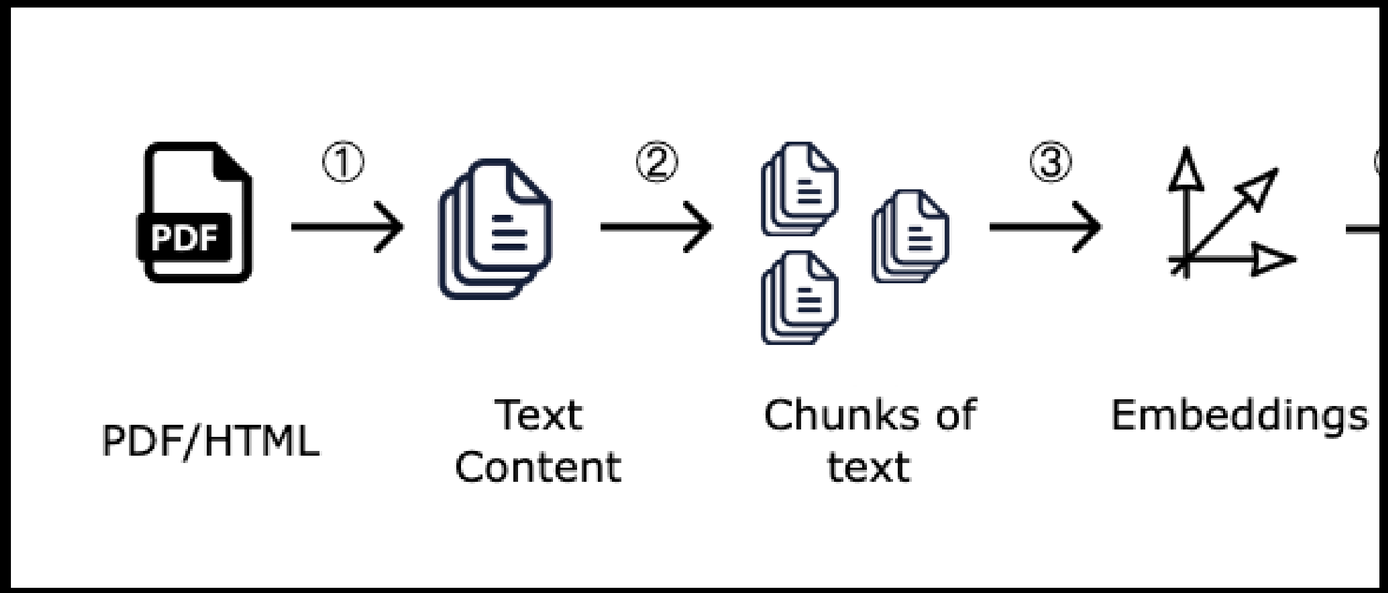
Gen
AI

Embeddings

Gen
AI

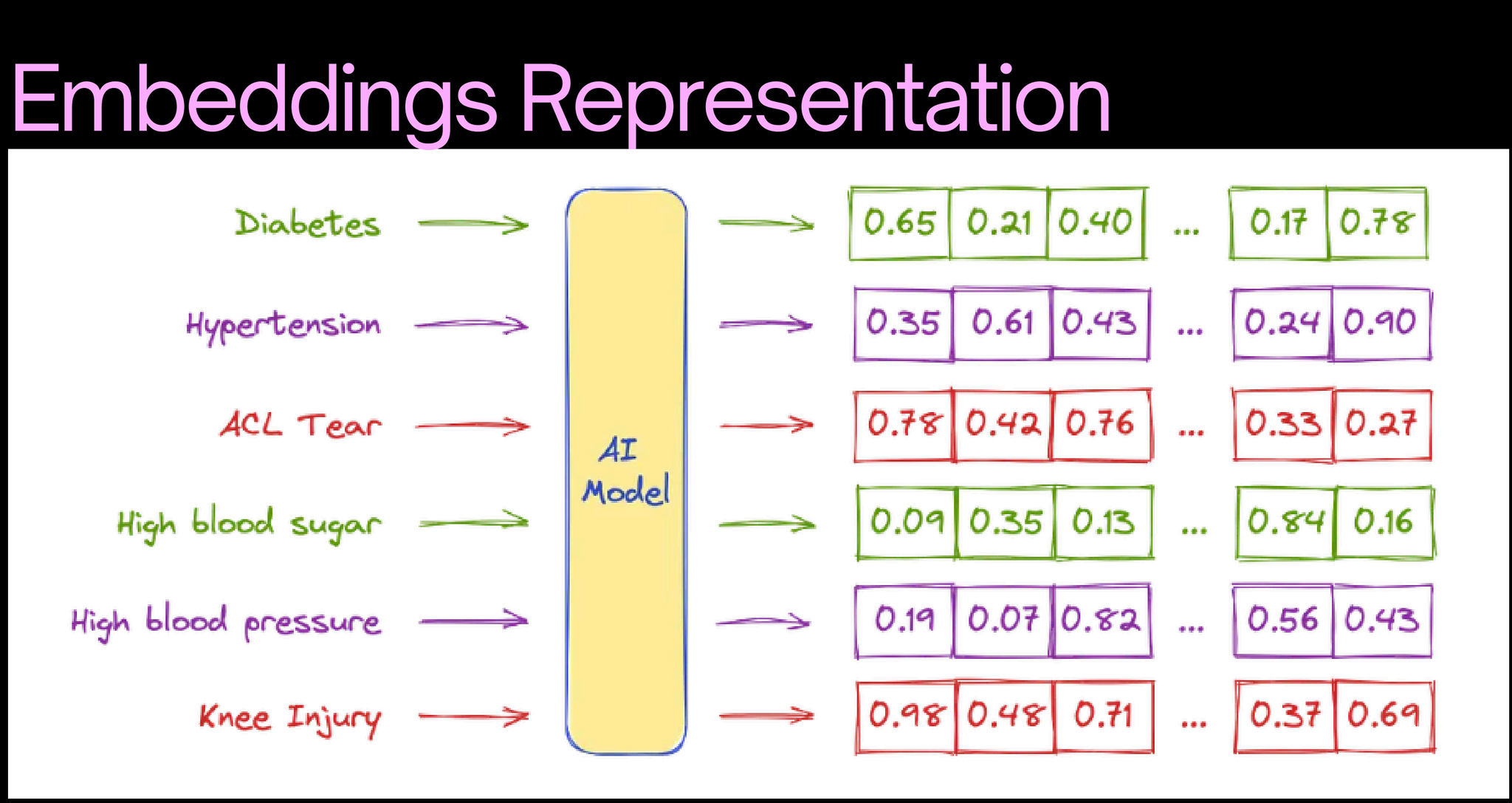
What are Embeddings?

Embeddings are numerical representations of text (or other data like images, audio, etc.) in a high-dimensional vector space.



Gen
AI

Embeddings Representation



Gen
AI

Embeddings Model

We will be using following embeddings model:

- text-embedding-3-large (Max Tokens: 8191)
- all-MiniLM-L6-v2 (Max Tokens: 256)

Gen
AI

For More Embeddings Model

Visit LeaderBoard: https://huggingface.co/spaces/mteb/leaderboard

Summary | Performance per Model Size | Performance over Time | Performance per Task Type | Performance per task | Performance per language | Task information

Filter...

Rank (Bor…	Model	Embedding D…	Max Tokens	Mean (T…	Mean (TaskT…	Bitext …	Classification	Clustering	Instruction R…	Multilabel Cl…
1	KaLM-Embedding-Gemma3-12B-2511	3840	32768	**72.32**	**62.51**	**83.76**	**77.88**	55.77	5.49	33.03
2	llama-embed-nemotron-8b	4096	32768	69.46	61.09	81.72	73.21	54.35	10.82	29.86
3	Qwen3-Embedding-8B	4096	32768	70.58	61.69	80.89	74.00	57.65	10.06	28.66
4	gemini-embedding-001	3072	2048	68.37	59.59	79.28	71.82	54.59	5.18	29.16
5	Qwen3-Embedding-4B	2560	32768	69.45	60.86	79.36	72.33	57.15	11.56	26.77
6	Octen-Embedding-8B	4096	32768	67.84	60.28	80.35	66.68	55.68	8.90	25.23
7	F2LLM-v2-14B	5120	40960	68.74	59.45	77.15	73.00	**60.91**	0.62	28.14
8	F2LLM-v2-8B	4096	40960	68.09	58.99	75.96	71.93	60.62	0.85	27.38
9	Seed1.6-embedding-1215	2048	32768	70.26	61.34	78.68	76.75	56.78	-0.02	**46.16**
10	F2LLM-v2-4B	2560	40960	67.06	58.25	74.49	70.73	59.53	2.25	26.58
11	jina-embeddings-v5-text-small	1024	32768	67.00	58.90	69.71	71.32	53.41	1.35	41.97

Gen
AI

OpenAI Embeddings Model: text-embedding-3-large

For Single Query Embeddings

```python
from langchain_openai import OpenAIEmbeddings
from dotenv import load_dotenv

load_dotenv()

embedding = OpenAIEmbeddings(model='text-embedding-3-large', dimensions=5)

result = embedding.embed_query("When Pakistan became nuclear power?")

print(result)
```

Gen
AI

OpenAI Embeddings Model: text-embedding-3-large

For Multiple Documents Embeddings

```python
from langchain_openai import OpenAIEmbeddings
from dotenv import load_dotenv
load_dotenv()

embedding = OpenAIEmbeddings(model='text-embedding-3-large', dimensions=5)
documents = [
    "Islamabad is the capital of Pakistan",
    "Karachi is the largest city of Pakistan",
    "Lahore is famous for its cultural heritage"
]

result = embedding.embed_documents(documents)
print(result)
```

Gen
AI

SentanceTransformers Embeddings Model: all-mpnet-base-v2

For Single Query Embeddings

```
from langchain_huggingface import HuggingFaceEmbeddings

embedding = HuggingFaceEmbeddings(model_name='sentence-transformers/all-MiniLM-L6-v2')

result = embedding.embed_query("When Pakistan became nuclear power?")

print(result)
```

Gen
AI

SentanceTransformers Embeddings Model: all-mpnet-base-v2

For Multiple Documents Embeddings

```python
from langchain_huggingface import HuggingFaceEmbeddings

embedding = HuggingFaceEmbeddings(model_name='sentence-transformers/all-MiniLM-L6-v2')

documents = [
    "Islamabad is the capital of Pakistan",
    "Karachi is the largest city of Pakistan",
    "Lahore is famous for its cultural heritage"
]

vector = embedding.embed_documents(documents)

print(vector)
```

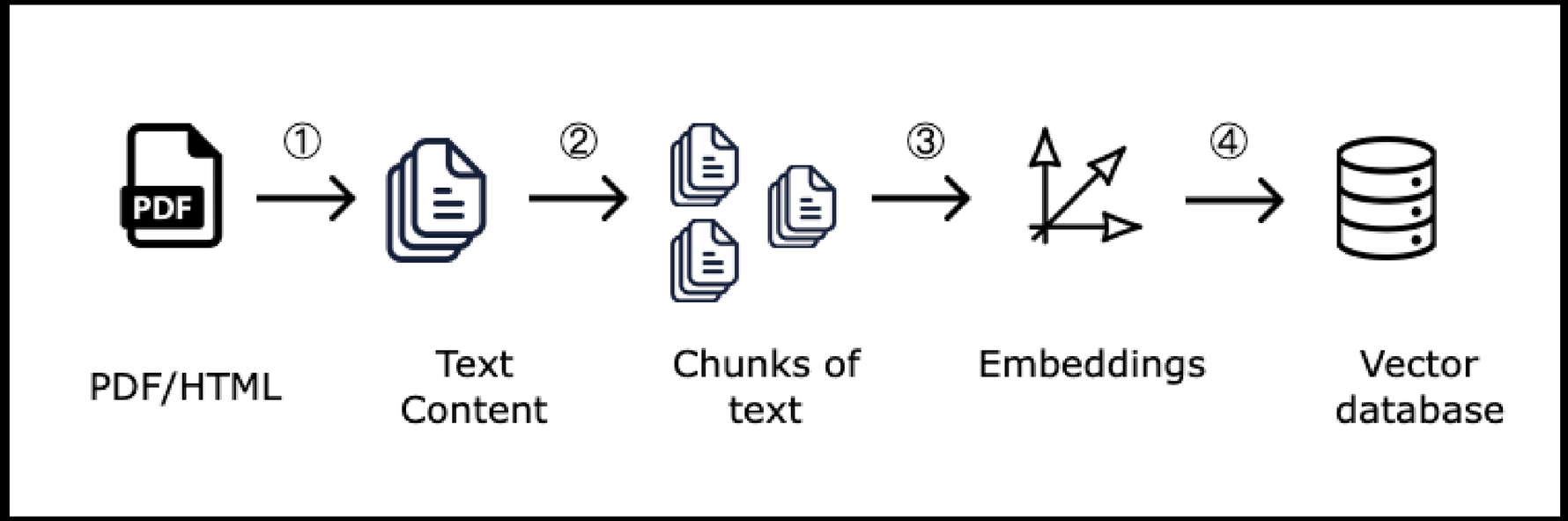
Gen
AI

Vector Database

Gen
AI

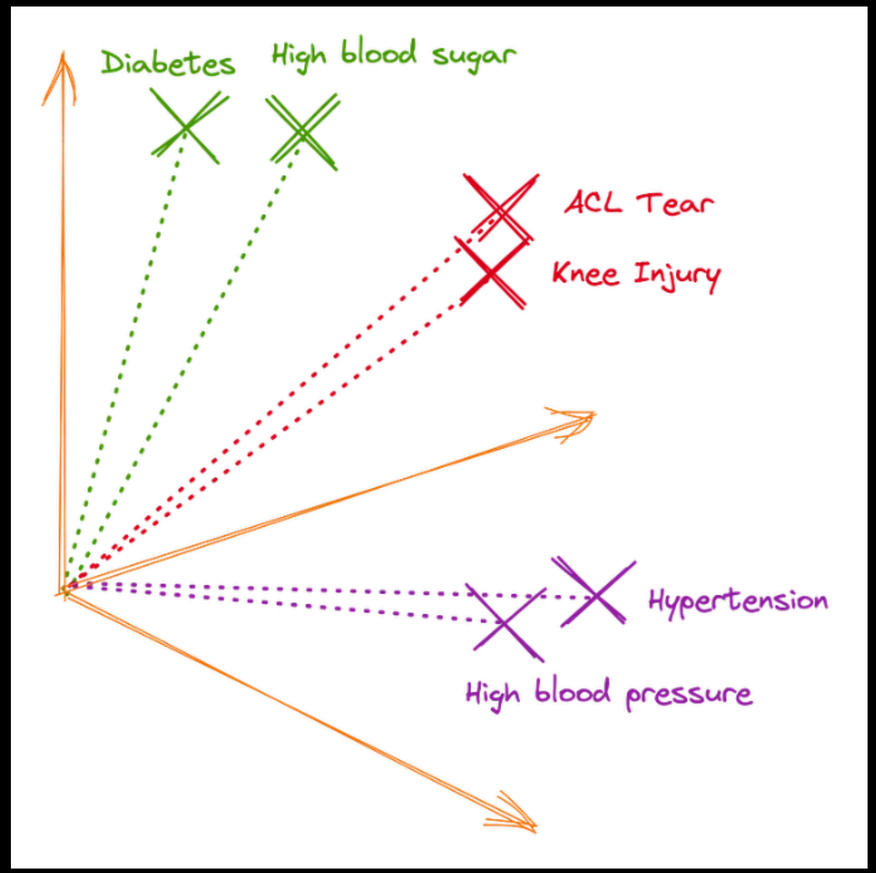
What is VectorDB?

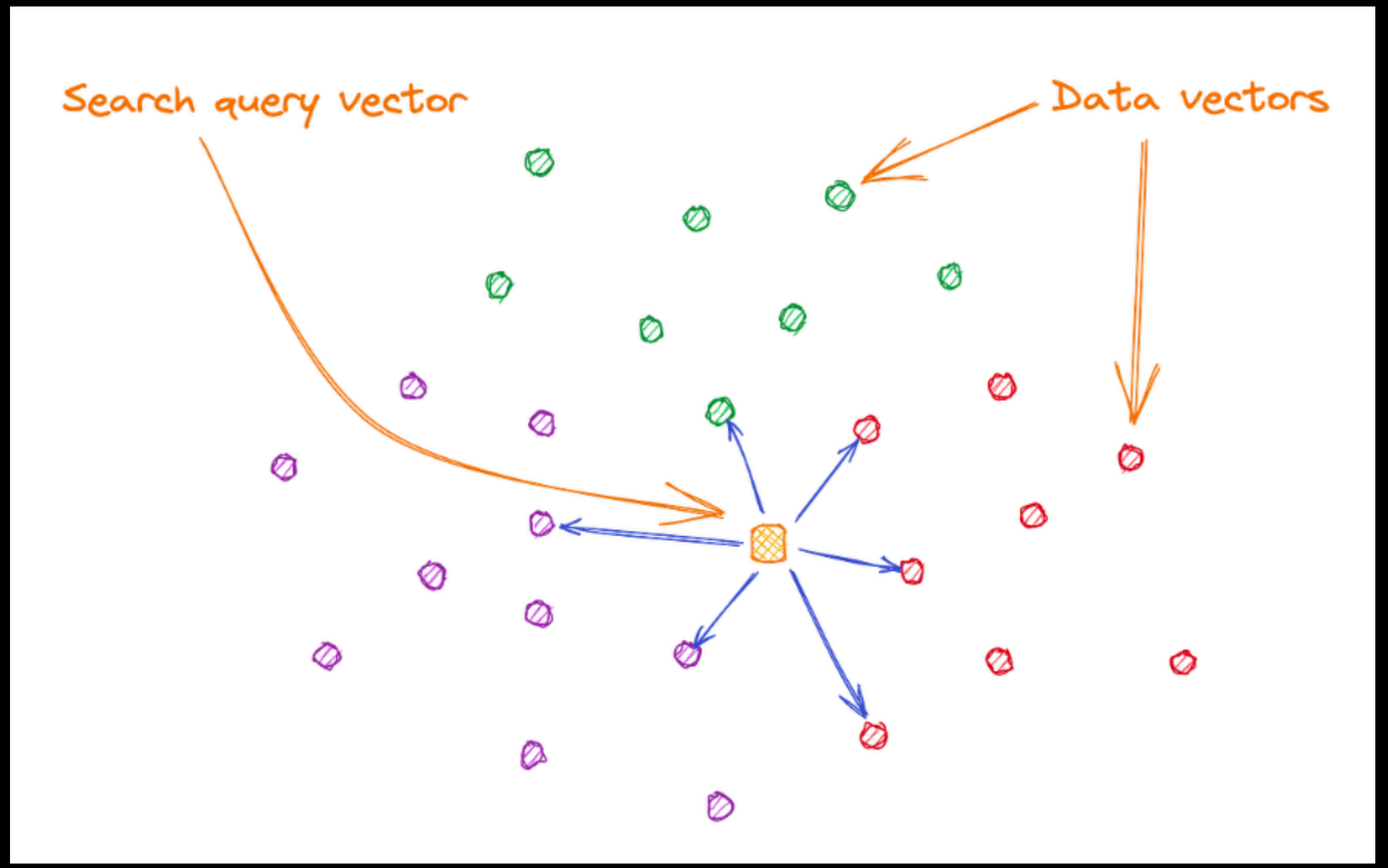
A Vector Database (Vector DB) is a database optimized for storing and querying vector embeddings instead of traditional rows/columns.



Gen
AI

Embedings Representation in VectorDB

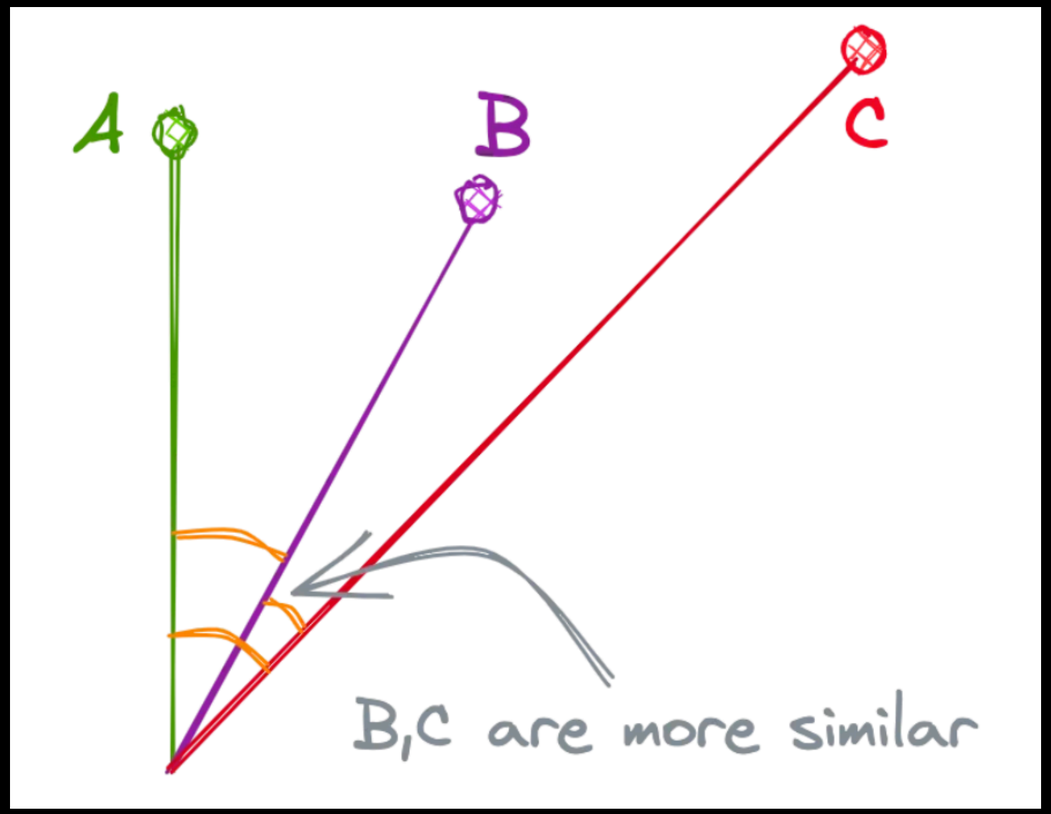




Gen
AI

Similarity Search Algorithm

There are few similarity search algorithms but Cosine Similarity is best



Gen
AI

Optimizing Similarity Search

Most of the VectorDB uses Hierarchical Navigable Small World graph (HNSW) algorithm for better similarity search.

Why do we need Optimizing Similarity Search?

If there are one million data vectors, there will be one million similarity calculations, and the most similar data vectors will be returned as the result. However, when dealing with millions or billions of records, this can severely impact performance due to the large number of calculations and memory required.

Gen
AI

Hierarchical Navigable Small World graph (HNSW)

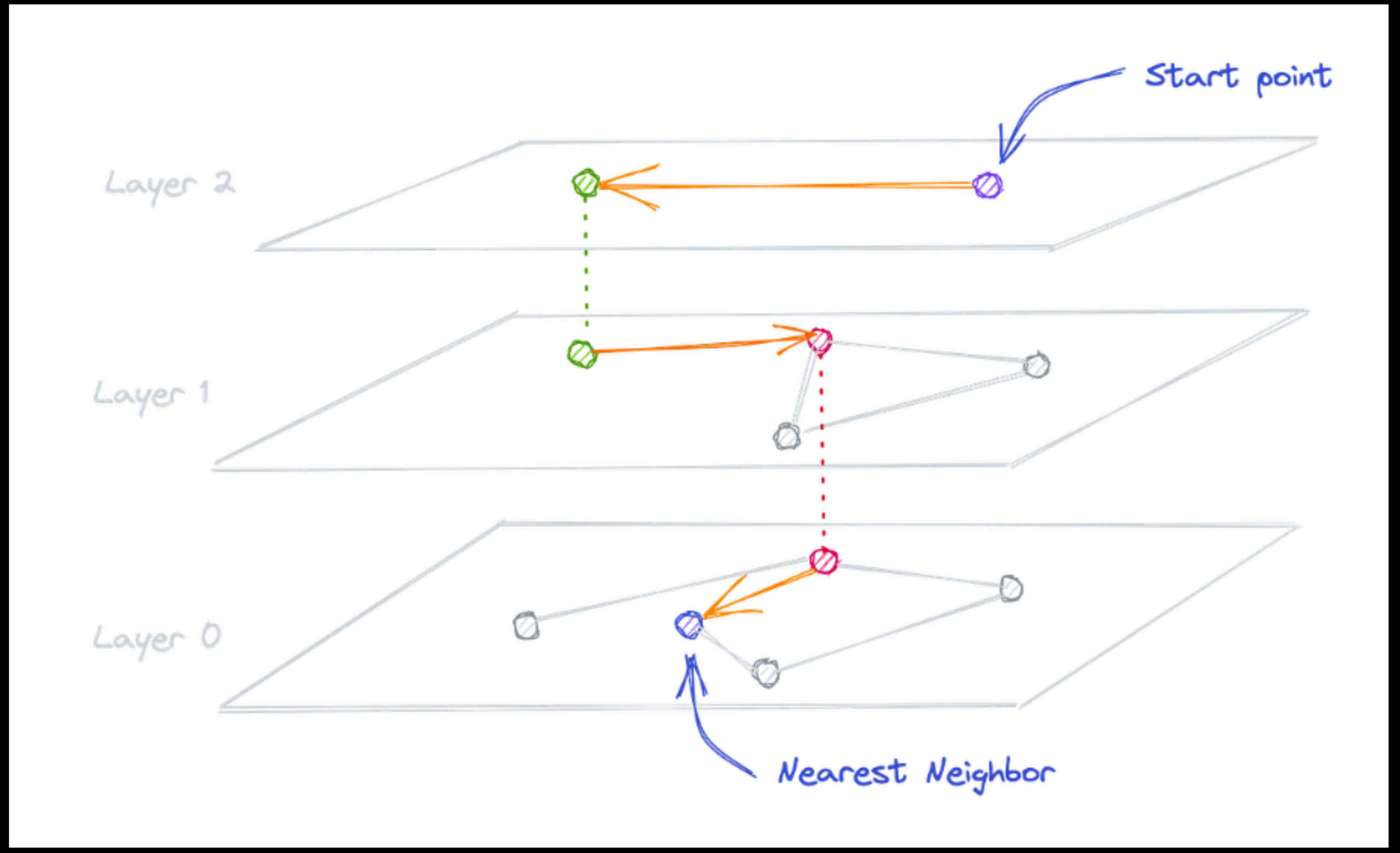
This optimization combines two different methods:

1. Probability Skip List
2. Navigable Small World Graph.

It creates a multi-layer graph from the given data, which helps the algorithm quickly find approximate nearest neighbors.

Gen
AI

Hierarchical Navigable Small World graph (HNSW)



Gen
AI

Benifits Of VectorDB Over Traditional Databases

There are two main benifits of using VectorDB:

- Fast Retrieval
- Optimized Storage

Gen
AI

ChromaDB - Create Instance

```python
from langchain_chroma import Chroma

vector_store = Chroma(
    collection_name="example_collection",
    embedding_function=embeddings,
    # Where to save data locally, remove if not necessary
    persist_directory="./chroma_langchain_db",
)
```

Gen
AI

CRUD Operations on VectorDB

Gen
AI

ChromaDB - Add Documents → 1

```python
from uuid import uuid4
from langchain_core.documents import Document

document_1 = Document(
    page_content=("I had chocolate chip pancakes and "
    "scrambled eggs for breakfast this morning."),
    metadata={"source": "tweet"},
    id=1,
)
document_2 = Document(
    page_content=("The weather forecast for tomorrow "
    "is cloudy and overcast, with a high of 62 degrees."),
    metadata={"source": "news"},
    id=2,
)
```

Gen
AI

ChromaDB - Add Documents → 2

```
documents = [
    document_1,
    document_2
]
uuids = [str(uuid4()) for _ in range(len(documents))]
vector_store.add_documents(documents=documents, ids=uuids)
```

Gen
AI

ChromaDB - Update Documents → 1

```
updated_document_1 = Document(
    page_content=("I had chocolate chip pancakes "
    "and fried eggs for breakfast this morning."),
    metadata={"source": "tweet"},
    id=1,
)
updated_document_2 = Document(
    page_content=("The weather forecast for tomorrow "
    "is sunny and warm, with a high of 82 degrees."),
    metadata={"source": "news"},
    id=2,
)
```

Gen
AI

ChromaDB - Update Documents → 1

```
# Update single document
vector_store.update_document(document_id=uuids[0], document=updated_document_1)

# You can also update multiple documents at once
vector_store.update_documents(
    ids=uuids[:2], documents=[updated_document_1, updated_document_2]
)
```

Gen
AI

ChromaDB - Delete Documents

```
updated_document_1 = Document(
    page_content=("I had chocolate chip pancakes "
    "and fried eggs for breakfast this morning."),
    metadata={"source": "tweet"},
    id=1,
)


vector_store.delete(ids=[1])
```

Gen
AI

ChromaDB - Search Upon Query

```
results = vector_store.similarity_search(
    "LangChain provides abstractions to make working with LLMs easy",
    k=2,
    filter={"source": "tweet"},
)
for res in results:
    print(f"* {res.page_content} [{res.metadata}]")
```

Gen
AI

ChromaDB - Search Upon Query with Score

```
results = vector_store.similarity_search_with_score(
    "Will it be hot tomorrow?", k=1, filter={"source": "news"}
)
for res, score in results:
    print(f"* [SIM={score:3f}] {res.page_content} [{res.metadata}]")
```

To Learn More About VectorDBs Integeration in Langchain Visit:
https://docs.langchain.com/oss/python/integrations/vectorstores

Gen
AI

Exercise

Do this exercise using an open-source embedding model from Hugging Face. Use different documents and queries to test similarity search.

1. Choose a small open-source embedding model (e.g., "sentence-transformers/all-MiniLM-L6-v2").
2. Create a new set of documents (e.g., Pakistani cities, historical places, or favorite sports).
3. Write a query and find the most relevant document using cosine similarity.
4. Print the query, top document, and similarity score.

Gen
AI

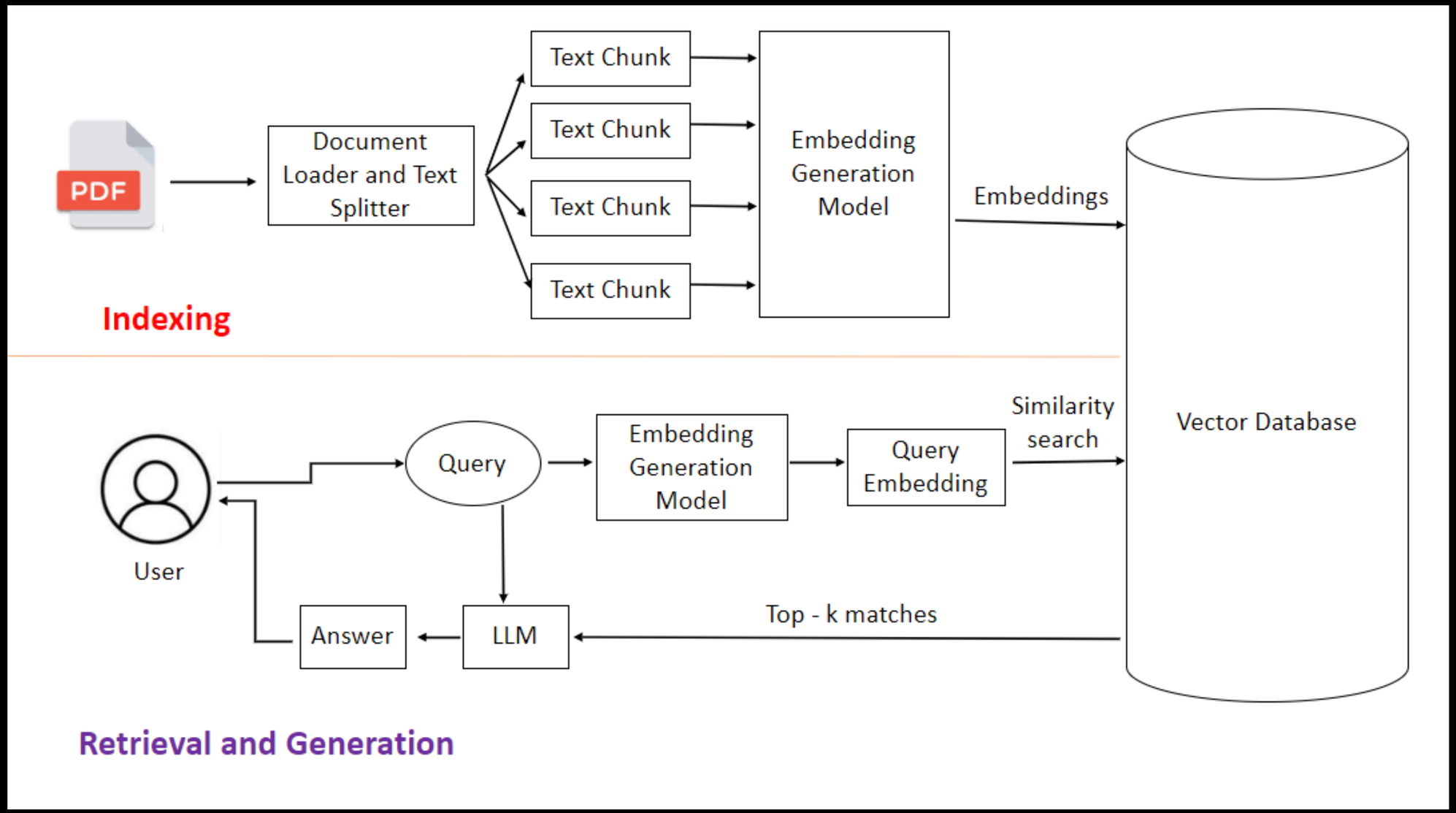
Use Case: Smart Product Assistant

Goal: An AI chatbot that answers user questions directly from official product manuals using Retrieval-Augmented Generation (RAG).

Purpose and benefit: Enhance user experience

Gen AI

Use Case: Architecture Diagram



Gen
AI

Tools

Gen
AI

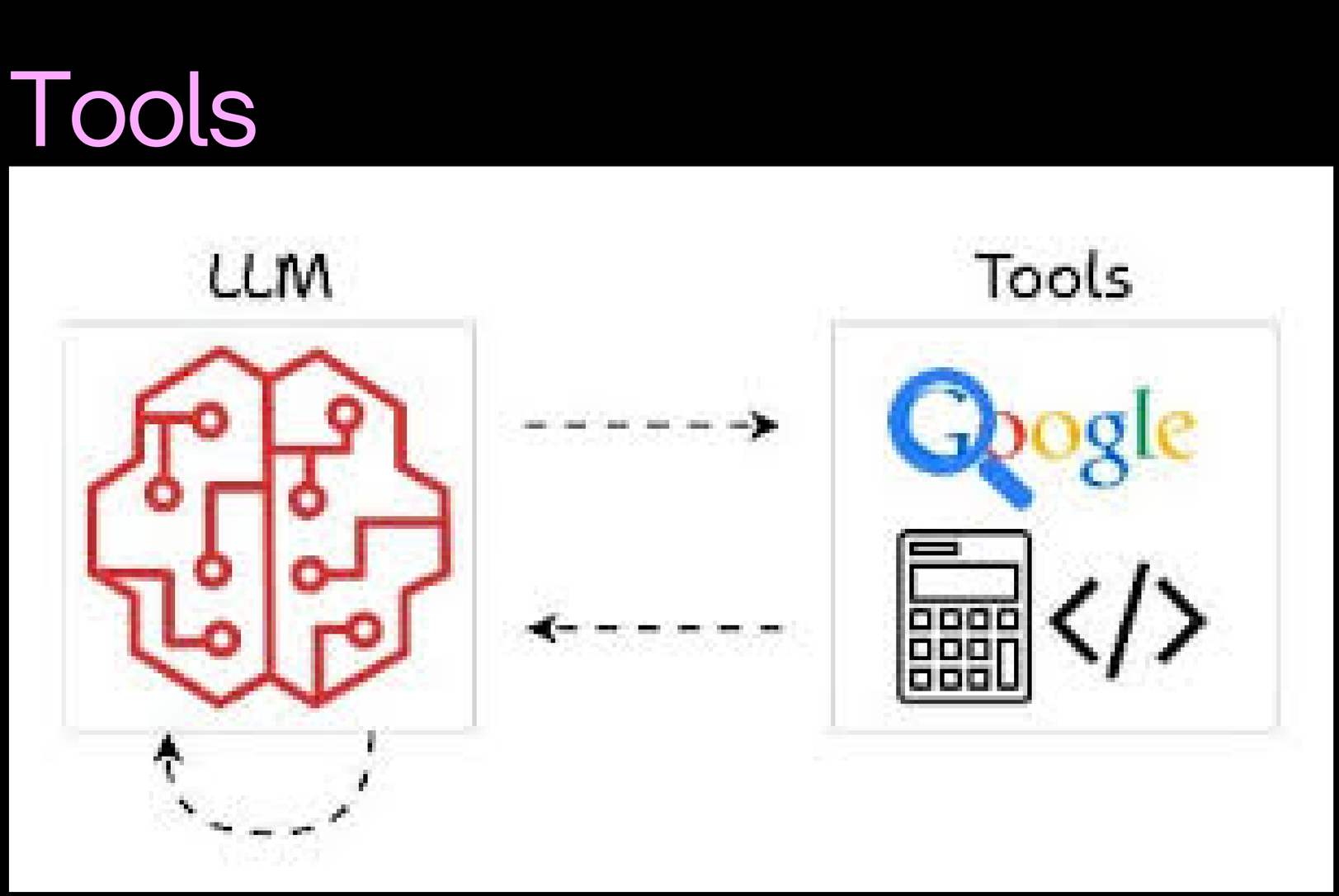
What is Tool?

A tool is just a python function that LLM can understand and call when needed

Why do we need Tools?

As LLMs can’t perform actions so, tools enable LLMs to perform actions

Tools



Gen
AI

Types of Tools in Langchain

There are two kinds of tools support in langchain, as follows:

- Built-in Tool
- Custom Tool

Gen
AI

Built-in Tool

LangChain provides built-in tools to let LLMs interact with real systems.

Some of them are below:

- DuckDuckGo
- ShellTool
- PythonREPLTool

Explore more tools:
https://docs.langchain.com/oss/python/integrations/tools

Gen
AI

Built-in Tool - DuckDuckGo Search

```
from langchain_community.tools import DuckDuckGoSearchRun

search_tool = DuckDuckGoSearchRun()

results = search_tool.invoke('top news in pakistan today')

print(results)
```

Gen
AI

Built-in Tool - DuckDuckGo Search - Multiple Results

```python
from langchain_community.tools import DuckDuckGoSearchResults

search_tool = DuckDuckGoSearchResults(num_results=5)
results = search_tool.invoke("Top today AI News")

entries = results.split("snippet: ")[1:]

for i, entry in enumerate(entries, start=1):
    parts = entry.split(", title: ")
    snippet = parts[0].strip()
    title, link = parts[1].split(", link: ")

    print(f"\nResult {i}")
    print("Title:", title.strip())
    print("Snippet:", snippet.strip())
    print("Link:", link.strip())
```

Gen
AI

Built-in Tool - ShellTool

```
from langchain_community.tools import ShellTool

shell_tool = ShellTool()

results = shell_tool.invoke('ls')

print(results)
```

Gen
AI

Built-in Tool - PythonREPLTool

```
from langchain_experimental.tools import PythonREPLTool
tool = PythonREPLTool()

result = tool.run("print(2 + 3)")
print(result)
```

Gen
AI

Custom Tool

A custom tool is a tool that you define yourself

When you will move to custom tool option?

- If you want to call your own API
- If you want to make your custom business logic

Gen
AI

Custom Tool - Tool Decorator

```python
from langchain_core.tools import tool
@tool
def multiply(a: int, b:int) -> int:
    """Multiply two numbers"""
    return a*b
```

Gen
AI

Custom Tool - StructuredTool → 1

```python
from langchain_core.tools import StructuredTool
from pydantic import BaseModel, Field

class MultiplyInput(BaseModel):
    a: int = Field(required=True, description="The first number to multiply")
    b: int = Field(required=True, description="The second number to multiply")

def multiply_func(a: int, b: int) -> int:
    return a * b
```

Gen
AI

Custom Tool - StructuredTool → 2

```
multiply_tool = StructuredTool.from_function(
    func=multiply_func,
    name="multiply",
    description="Multiply two numbers",
    args_schema=MultiplyInput
)
result = multiply_tool.invoke({'a':3, 'b':5})
```

Gen
AI

Custom Tool - StructuredTool → 3

```
print(result)
print(multiply_tool.name)
print(multiply_tool.description)
print(multiply_tool.args)
```

Gen
AI

Tool-Augmented Chatbot for Imtiaz Super Store

Gen
AI

Use Case: E-Commerce Personal Assistant

Imtiaz Super Store wants to build an **AI-powered shopping assistant** that goes beyond chat and performs **real business operations** using live product data and customer loyalty information.
This chatbot acts as a **personal shopping assistant**, helping users:

- Discover products
- Apply loyalty points intelligently
- Get final, real-time pricing
- Check their loyalty balance

Gen
AI

End of Class

-